

AXON-RFC-006 Appendix A: easynet.pages

Technical design, algorithms, and implementation plan for the first reference state machine of stateful EasyNet

Silan Hu

National University of Singapore

Draft v0.2, April 28, 2026

Abstract

This appendix specifies `easynet.pages`, the simplest `state_type` that exercises the full set of normative primitives defined in AXON-RFC-006 main. A page is a state object that carries identity, canonical content, and version under the rules of §§1–7 of the main body, and exposes that state through two views: `human_view` (an HTML rendering for browsers) and `agent_view` (a structured JSON metadata document — canonical fields, ability surface, receipt-history pointer, and an optional pointer to the page’s `actions.*` namespace if the owner exposed one).

What changed in v0.2. The agent-actionable surface (EAL fragments, missions, form maps) is no longer embedded into `agent_view` as a static field. Instead, when a page’s owner wishes to expose action-style discovery, they implement abilities in the `actions.*` namespace, exactly as described in Appendix B for `web_apps`. Most pages are simple documents and do not need actions; `agent_view`’s `actions` block is then absent or empty. This brings pages and `web_app` into a single agent-discovery convention.

The appendix covers four layers of detail. First, the state machine: the field set, the four-state space, and the seven transitions. Second, the wire schemas of every transition’s arguments and receipt body. Third, the algorithms: canonical projection, JCS canonicalisation, hash, ETag, optimistic concurrency, and view projection, each with a worked byte-level example. Fourth, the implementation plan: a module map for `EasyNet-Cli`, a five-PR slice plan (P-A1 through P-A5), and a complete end-to-end worked example following one page from its first `pages.create` through a Hub-rendered HTML response and an agent-readable URA fetch. Appendix B (the `easynet.web_app` pressure test) and the main body’s thirteen `TR-INV-*` invariants are referenced throughout but not redefined; this document is a *reference implementation specification*, not a re-derivation of the protocol.

Contents

1	Goals and non-goals	2
1.1	Goals	2
1.2	Non-goals	3
2	Relation to the normative core	3
3	State machine	4
3.1	<code>state_type</code> and <code>state_key</code>	4
3.2	Canonical fields	5
3.3	State space	5
3.4	Lifecycle diagram	6
4	Wire schemas	6
4.1	<code>pages.create@v1</code>	7
4.2	<code>pages.publish@v1</code>	7

4.3	pages.update@v1	7
4.4	pages.archive@v1, pages.unpublish@v1, pages.delete@v1	8
4.5	pages.get@v1 (query)	8
4.6	pages.list@v1 (query)	8
5	Algorithms	9
5.1	Canonical projection π	9
5.2	Canonical serialisation σ (JCS)	9
5.3	canonical_state_hash h	9
5.4	Worked byte example	9
5.5	Optimistic concurrency: ETag	10
5.6	View projection algorithms	10
5.6.1	human_view (HTML)	10
5.6.2	agent_view (structured)	11
5.6.3	actions: optional pointer to a queryable namespace	12
6	Architecture	12
6.1	Module map (EasyNet-Cli)	12
6.2	Daemon wiring	13
6.3	Hub wiring	13
6.4	Storage layout	14
7	Implementation plan	14
7.1	P-A1: BlobStore + StateStore skeletons	14
7.2	P-A2: pages.* ability registration + receipts	15
7.3	P-A3: view projection + EAL hints	15
7.4	P-A4: Hub binary	16
7.5	P-A5: agent_view enrichment + receipt history endpoint	16
7.6	Hub envelope minting	16
8	End-to-end worked example	17
8.1	Setup	17
8.2	Author and publish	17
8.3	Browser fetch via Hub	17
8.4	Peer agent fetches via URA	18
8.5	Audit replay	19
9	Open questions deferred to RFC-006-A v2	19
9.1	Coverage check	20

1 Goals and non-goals

1.1 Goals

[normative] This appendix specifies an implementation of `easynet.pages` that:

1. Conforms to every **[normative]** clause of AXON-RFC-006 main (§§1–7) and discharges every applicable TR-INV-* from the companion Appendix B.
2. Provides four canonical transitions (`create`, `publish`, `update`, `archive`, `delete`) and two query abilities (`get`, `list`).

3. Stores page content as a content-addressed blob, keyed by SHA-256, never embedded in canonical state (CSH-INV-2).
4. Exposes both `human_view` (HTML rendered from the content blob) and `agent_view` (a typed JSON document containing canonical fields, ability surface, receipt history pointer, and an optional pointer to the page's `actions.*` namespace if the owner exposed one) for every published page.
5. Reuses the same `easynet-pages-author` agent skill that Appendix B seeds. The skill teaches an agent how to author both pages (this appendix) and `web_apps` (Appendix B); the structural overlap on `pages.create` / `publish` / `update` / `delete` makes one bundled skill sufficient. The seeder lives in P-A5 and is described in detail in Appendix B §3.4.
6. Is implementable end-to-end in five PR-sized slices (§7).

1.2 Non-goals

[derived] The following are deliberately out of scope:

- **webapp** (the long-running, runtime-overlay pressure test). Covered by Appendix B.
- **cross-realm or cross-node hubs**. Phase 1 ships a single-node Hub that reverse-proxies pages owned by the same daemon. Multi-realm hubs are deferred to RFC-006-A v2.
- **authenticated human callers** (`principal/human-auth`). Phase 1 supports `principal/human-anon` only.
- **HTML sanitisation pipelines and CSP policies beyond a fixed default**. Owners author HTML; the Hub serves it under a single hard-coded CSP. A sanitisation/template layer is RFC-006-A v2.
- **TLS termination, rate limiting, multi-instance Hub fleets**. The Hub is an axum process binding a single port; deployment-grade hardening is operations work, not protocol work.

2 Relation to the normative core

This appendix is a *reference implementation* of the normative core in AXON-RFC-006 main. It does not redefine any primitive. The mapping is:

Main clause	This appendix's instantiation
§1 State Object	§3: pages state_type, key, lifecycle.
§1.5 SO-INV-1 immutability	state_key (realm, owner, slug) is immutable post-create.
§2 Canonical State Hash	§5: explicit projection π , JCS, SHA-256 example.
§2 CSH-INV-2 content-as-blob	§6.4: blob layout; canonical state holds only content_hash.
§3 Receipt Classes	§4: every transition's receipt body shown.
§3 RC-INV-2 optimistic concurrency	§5.5: ETag = first 16 bytes of canonical hash.
§4 Transition Ability	§4: transition_ids for every page transition.
§5 Authority + delegation	Phase 1: owner_agent signs locally; no delegated execution for pages.
§5 principal/ caller	§7.6: Hub mints human-anon URIs per request.
§6 View Projection	§5.6: human_view and agent_view algorithms.
§7 Receipt Indexing	§6.4: state_key index in addition to receipt_id.

Reading discipline. A statement labelled **[normative]** in this appendix is binding on the pages implementation only; it does not propose a new protocol-level rule. Where this document appears to extend the protocol, it does so by instantiating an already-**[normative]** clause of the main body.

3 State machine

3.1 state_type and state_key

[normative]

```
state_type: easynet.page

state_key:
    (realm: string, owner_agent_uri: string, slug: string)

    realm          Logical namespace. Phase 1: a single realm per
                    daemon, named in config (default "default").
    owner_agent_uri Canonical authority. The agent that signs every
                    CanonicalReceipt for this page. Embedded in the
                    key (not a field) per main §1.3.
    slug           Owner-namespaced kebab-case identifier
                    (^[a-z0-9][a-z0-9-]{0,62}[a-z0-9]$).

URA form:
    easynet:///r/<realm>/agent/<owner>/page/<slug>

ABNF (slug):
    slug      = head [ *body tail ]
    head      = ALPHA / DIGIT
    body      = ALPHA / DIGIT / "-"
    tail      = ALPHA / DIGIT
    ALPHA     = %x61-7A          ; lowercase ASCII letters only
```

The slug grammar deliberately rejects uppercase, dots, slashes, and internationalised characters.

Two reasons: the slug appears in URLs with no escape pass; and case-sensitive collisions (**Hello** vs **hello**) are a rich source of admin confusion. RFC-006-A v2 may relax this.

3.2 Canonical fields

[normative] Per CSH-INV-2 (main §2.5), large content is held as a hash. The canonical state of a page is therefore a small map:

```
canonical_fields(page) = {
    manifest_hash:    bytes(32),    // SHA-256 of the manifest
                                // sub-document (title,
                                // description, content_type,
                                // visibility); see §3.3
    content_hash:     bytes(32),    // SHA-256 of the body bytes,
                                // null/zero in Draft and after
                                // archive/delete
    state_value:      string,        // "Draft" | "Published"
                                // | "Archived" | "Deleted"
    visibility:       string,        // "private" | "realm" | "public"
    version:          uint64         // monotonic; +1 per
                                // CanonicalReceipt
}
```

The schema annotates every field with `canonical:true`. The following *operational* fields exist in Phase 1 but are *never* part of canonical projection π (§5):

```
operational_fields(page) = {
    last_view_at:      timestamp,    // observation only
    view_count_estimate: uint64,      // best-effort
    last_caller_class: string|null    // "browser" | "bot" | ...
}
```

These are computed from `OperationalReceipts` emitted by the Hub during view projection (§5.6); they are useful for the `agent_view`'s audit summary but never enter `canonical_state_hash`.

3.3 State space

state_value	Meaning
Draft	Created but not yet published. <code>content_hash</code> may be null or set; not served by Hub.
Published	Live. <code>content_hash</code> non-null. Served by Hub if visibility allows.
Archived	Previously Published; not served by Hub. <code>content_hash</code> retained for history.
Deleted	Terminal. <code>content_hash</code> cleared. No further transitions accepted.

Allowed canonical transitions (single canonical state space, no runtime overlay — pages have no long-running serve process, unlike `web_app`):

Transition	Pre → Post	Notes
<code>pages.create@v1</code>	$\emptyset \rightarrow \text{Draft}$	Mints state key.
<code>pages.publish@v1</code>	$\text{Draft} \rightarrow \text{Published}$	Requires non-null <code>content_hash</code> .
<code>pages.update@v1</code>	$\text{Published} \rightarrow \text{Published}$	<code>version+1</code> ; <code>content_hash</code> and/or <code>manifest_hash</code> change.
<code>pages.archive@v1</code>	$\text{Published} \rightarrow \text{Archived}$	Reversible by re-publish.
<code>pages.unpublish@v1</code>	$\text{Archived} \rightarrow \text{Draft}$	Allows resume of editing.
<code>pages.delete@v1</code>	$* \rightarrow \text{Deleted (terminal)}$	Clears <code>content_hash</code> .

Query abilities (no state mutation, no `CanonicalReceipt`, only `OperationalReceipt` for audit):

Ability	Returns
<code>pages.get@v1</code>	full canonical projection + content blob handle
<code>pages.list@v1</code>	list of (slug, <code>state_value</code> , version) for one owner

Invariant — PA-INV-1

A page **Draft** or **Archived** **MUST NOT** be served by any Hub regardless of visibility. The Hub discriminates on `state_value` *before* consulting visibility; visibility gates only Published pages.

Why PA-INV-1. A draft is by definition unfinished; serving it because visibility happened to be public would surface work-in-progress as published content. Archived pages are deliberately un-served — they exist for history, not access.

3.4 Lifecycle diagram



4 Wire schemas

Every transition's `ability` input schema and the `receipt_schema` for its `CanonicalReceipt` are listed below. JSON Schema is abbreviated (types, required keys only); the canonical schemas

live in `abilities/pages.<verb>.ability.toml`.

4.1 `pages.create@v1`

```
args:
{
  slug:          string (matches §3.1 slug ABNF, required)
  manifest:      {
    title:        string,          required
    description:  string|null,
    content_type: "text/html" | "text/markdown",
                  required
    visibility:   "private" | "realm" | "public",
                  required
  }
  initial_body:  bytes|null        // utf8 or base64 per content_type;
                                  // null leaves the page empty draft
}
```

CanonicalReceipt body extends the main-§3.1 base with:

```
{
  page_state_key:   (realm, owner, slug),
  pre_state_hash:   zero(32),      // creation: pre is empty
  post_state_hash:  sha256(jcs(canonical_fields)),
  state_version:    1,
  manifest_hash:    sha256(jcs(manifest)),
  content_hash:     sha256(initial_body) if initial_body else zero(32),
  blob_handles:     [content_hash] if content_hash != 0 else []
}
```

4.2 `pages.publish@v1`

```
args:
{
  slug:    string,
  // optional: replace body before publishing
  body:    bytes|null,
  reason:  string|null // free-form, recorded in receipt
}
```

precondition: `state_value == Draft AND content_hash != 0`
 (or ``body`` is provided and non-empty)

CanonicalReceipt body:

```
{
  page_state_key:   ...,
  pre_state_hash:   canonical_state_hash(Draft snapshot),
  post_state_hash:  canonical_state_hash(Published snapshot),
  state_version:    prev+1,
  content_hash:     sha256(new body),
  blob_handles:     [content_hash],
  reason:           args.reason
}
```

4.3 `pages.update@v1`

```

args:
{
  slug:      string,
  manifest:  manifest|null,    // partial update permitted;
                                // null = unchanged
  body:      bytes|null,      // null = unchanged
  reason:    string|null
}
precondition: state_value == Published AND
              (manifest != null OR body != null)

CanonicalReceipt body:
{
  page_state_key:    ...,
  pre_state_hash:    h_prev,
  post_state_hash:   h_next,
  state_version:     prev+1,
  manifest_hash:     unchanged or new,
  content_hash:      unchanged or new,
  changed_fields:    ["manifest_hash"|"content_hash", ...],
  reason:            args.reason
}

```

4.4 pages.archive@v1, pages.unpublish@v1, pages.delete@v1

These three follow the same template: small `args` (`slug`, `reason`), a `CanonicalReceipt` with correctly-set `pre_state_hash` / `post_state_hash` / `state_version`. `pages.delete@v1` additionally clears `content_hash` and *decrements* the blob's ref-count (Phase 1 ref-count is a stub; see RFC-007).

4.5 pages.get@v1 (query)

```

args:
{
  slug:    string,
  version: uint64|null    // null = current
}
returns:
{
  state_key:    ...,
  state_value:  string,
  canonical:    canonical_fields,
  body:         bytes|null,    // null if Deleted/Archived
                                // when caller is unauthorised;
                                // omitted entirely if visibility
                                // gates the caller out

  version:      uint64,
  canonical_state_hash: bytes(32),
  etag:         bytes(16)      // §5.4
}

```

`pages.get@v1` produces an `OperationalReceipt` recording the read, the caller URI, and (when from a Hub) the `caller_context` (main §3.6).

4.6 pages.list@v1 (query)

```
args:
{
  owner:      string,
  state_filter: ["Draft"|"Published"|...] | null,
  limit:      uint <= 1000,
  cursor:     string|null
}
returns:
{
  pages:      [{slug, state_value, version, etag, last_modified}],
  next_cursor: string|null
}
```

5 Algorithms

5.1 Canonical projection π

[normative] Per main §2.1, π is schema-driven, not hand-rolled. The pages schema lists exactly five fields with `canonical:true`: `manifest_hash`, `content_hash`, `state_value`, `visibility`, `version`. $\pi(v)$ projects v onto exactly those keys, in schema-declared order.

```
fn project_pages(state: &PageState) -> Map<String, Value> {
  let mut out = BTreeMap::new();
  out.insert("manifest_hash".into(), bytes_to_hex(&state.manifest_hash));
  out.insert("content_hash".into(),  bytes_to_hex(&state.content_hash));
  out.insert("state_value".into(),   state.state_value.clone().into());
  out.insert("visibility".into(),    state.visibility.clone().into());
  out.insert("version".into(),       state.version.into());
  out // BTreeMap iterates in lexicographic order — deterministic
}
```

Cost. $O(1)$ time and space (five fields, fixed sizes).

5.2 Canonical serialisation σ (JCS)

[normative] σ is RFC 8785 (JSON Canonicalization Scheme). For pages this is a straight subset: object keys lexicographically sorted (already guaranteed by `BTreeMap`), no extraneous whitespace, hex strings for byte fields, integers without leading zeros, no floats. JCS for the pages projection is therefore a deterministic compaction of the projected JSON.

5.3 canonical_state_hash h

[normative] $h(b) = \text{SHA256}(b)$, fixed in Phase 1.

5.4 Worked byte example

A page in Published state with the following canonical projection:

```
{
  "content_hash": "abf0c1...c8f3", // 32 bytes hex
  "manifest_hash": "1d4a7e...09e2",
  "state_value":  "Published",
  "version":      3,
  "visibility":   "public"
}
```

After JCS:

```
{"content_hash":"abf0c1...c8f3","manifest_hash":"1d4a7e...09e2","state_value":"Published","version":3,"visibility":"public"}
```

`canonical_state_hash` = SHA256(above bytes) = 0xa3b91f7e...8d04 (32 bytes). The first 16 bytes serve as the ETag (§5.5).

5.5 Optimistic concurrency: ETag

[normative] Per RC-INV-2 (main §3.7) every canonical transition attempt declares its expected `pre_state_hash`. The Hub and HTTP clients work better with a 16-byte ETag than a 32-byte hex string; we therefore define:

$$\text{ETag}(p) = \text{Base64Url}(\text{canonical_state_hash}(p)[0..16])$$

The 128-bit prefix is collision-safe at the population sizes a single-realm Hub serves (birthday bound $\approx 2^{64}$ pages before any collision). On every `pages.update`, the caller supplies an `If-Match` header (or, in the EasyNet ability path, an `expected_etag` field). The receipt-validation layer:

1. Computes the page's current ETag.
2. Compares against `expected_etag`.
3. On mismatch: rejects the transition with `ConcurrencyConflict`, recording the conflict in an `OperationalReceipt` (so the audit log records what was attempted but not committed).

Note

ETag is a presentation of `canonical_state_hash`, not a substitute. Internal protocol logic uses the full 32-byte hash (RC-INV-2 cites `pre_state_hash`); the 16-byte ETag exists only at the external HTTP boundary.

5.6 View projection algorithms

5.6.1 human_view (HTML)

[normative]

```
fn human_view(p: &PageState, blob: &BlobStore) -> HttpResponse {
  if p.state_value != "Published" {
    return http_404();
  }
  if !visibility_allows(p.visibility, request.caller) {
    return http_404(); // never 403 — leaks existence
  }
  let body = blob.get(&p.content_hash)?;
  let csp = "default-src 'self'; script-src 'none'; \
    style-src 'unsafe-inline' 'self'; img-src 'self' data;; \
    object-src 'none'; base-uri 'none'; frame-ancestors 'none'";
  HttpResponse::ok()
    .content_type(p.manifest.content_type)
    .header("ETag", etag(p))
    .header("Cache-Control", "public, max-age=60, must-revalidate")
    .header("Content-Security-Policy", csp)
}
```

```

        .header("Referrer-Policy", "no-referrer")
        .header("X-Content-Type-Options", "nosniff")
        .body(body)
    }

```

Determinism. Per main TR-INV-6, `human_view` is a deterministic function of (p, blob) . The CSP, the cache header, and the response code are pure functions of `p` and the request's principal; nothing here depends on wall-clock time or load.

Why 404 (not 403) on visibility miss. A 403 confirms the page exists. For `visibility=private` this is a leak. The trade-off (private page indistinguishable from non-existent) is intentional.

5.6.2 agent_view (structured)

[normative]

```

fn agent_view(p: &PageState) -> Value {
    let surface = derive_ability_surface(p); // VP-INV-1
    let history = receipt_history_pointer(p); // §6.3
    let actions = actions_pointer(p);        // see "actions" below
    json!({
        "state_type": "easynet.page",
        "state_key": { "realm": p.realm,
                       "owner": p.owner,
                       "slug": p.slug },
        "ura": p.ura(),
        "canonical": {
            "state": p.state_value,
            "manifest": p.manifest, // small, in-line OK
            "content_hash": hex(p.content_hash),
            "visibility": p.visibility,
            "version": p.version,
            "etag": etag(p)
        },
        "ability_surface": surface,
        "history": {
            "versions": p.version,
            "latest_canonical_receipt_id": p.last_receipt_id,
            "receipt_log_ura": history
        },
        "actions": actions, // see below; null for simple pages
        "view_meta": {
            "generated_from": "(canonical, runtime=, caller_context)"
        }
    })
}

fn derive_ability_surface(p: &PageState) -> Vec<&'static str> {
    match p.state_value.as_str() {
        "Draft" => vec!["pages.publish@v1", "pages.update@v1",
                       "pages.delete@v1"],
        "Published" => vec!["pages.update@v1", "pages.archive@v1",
                            "pages.delete@v1"],
        "Archived" => vec!["pages.unpublish@v1", "pages.delete@v1"],
        "Deleted" => vec![],
        _ => vec![],
    }
}

```

Compliance with VP-INV-1. `ability_surface` is derived strictly from `p.state_value` and the static transition schema (the match block). It is not a snapshot of the ability registry; an ability that is registered but whose `pre_state` does not include `p.state_value` is invisible here.

5.6.3 actions: optional pointer to a queryable namespace

[normative] A simple document does not need an agent-actionable surface beyond “read me.” Most pages therefore set `actions: null` and that is the end of the story.

When a page DOES carry richer interaction — a contact form, a subscribe button, a schedule-meeting flow embedded in HTML — the owner exposes those actions through the `actions.*` ability namespace, exactly as `web_apps` do (Appendix B §6). `agent_view.actions` is then a small pointer:

```
{
  "namespace": "actions",
  "registered_verbs": ["actions.list_for_page",
                      "actions.get",
                      "actions.search"],
  "http_mirror_prefix": "/easynet-for-agents/",
  "discovery_hint":
    "POST /easynet-for-agents/list_for_page {\"page_path\": \"<path>\"}"
}
```

A peer agent reading this pointer calls `<owner>.actions.list_for_page` (or its HTTP mirror) to discover what it can do; `agent_view` itself never embeds the EAL/missions. The reasoning is identical to `web_app`: embedding actions in a static view forces bulk download and breaks per-intent specialisation. Pages reuse `web_app`’s design to keep the agent-facing convention identical across the two state types.

Single-call short form. For a page whose owner wants to expose exactly one action and prefers not to register any `actions.*` ability, an `eal_hint: <single-line-string>` field MAY be included inline in `agent_view` as a degenerate case — the single-line analog of a one-verb `actions.list_for_page` response. This short form is informative only; if both `actions` and `eal_hint` are present, `actions` is authoritative.

6 Architecture

6.1 Module map (EasyNet-Cli)

<code>src/runtime/state/</code>	← NEW (P-A1, P-A2)
<code>mod.rs</code>	<code>pub</code> use of types below
<code>store.rs</code>	<code>StateStore</code> trait + sled-backed <code>impl</code>
<code>hash.rs</code>	project / canonicalize_jcs / sha256
<code>receipt.rs</code>	<code>CanonicalReceipt</code> , <code>OperationalReceipt</code>
 <code>src/runtime/blob_store/</code>	← NEW (P-A1)
<code>mod.rs</code>	<code>BlobStore</code> trait
<code>fs.rs</code>	<code>FsBlobStore</code> (content-addressed directory tree, sha256 keys)
 <code>src/runtime/agents/pages_ability.rs</code>	← NEW (P-A2)

	register: create/publish/update/ archive/unpublish/delete/get/list
src/runtime/views/ mod.rs human.rs agent.rs	← NEW (P-A3) View trait human_view (CSP, blob fetch) agent_view (JSON projection)
src/bin/easynet-hub.rs	← NEW (P-A4) axum binary, path → URA → view
src/runtime/hub/ mod.rs router.rs caller.rs	← NEW (P-A4) path parsing, view selection principal/human-anon URI minting + caller_context extraction
src/runtime/eal_hint.rs	← NEW (P-A5) deterministic EAL templates

Design choice: sled for state and receipt index. sled is already a dependency in adjacent code; it gives us range queries (needed for `state_key`-prefix scans mandated by main §7.1) and atomic batches (needed to write a canonical receipt + state mutation in one go). RFC-006 main does not pin a backend; if sled is later replaced (e.g. by SQLite), the `StateStore` trait stays.

Design choice: filesystem blob store. Phase 1 stores blobs at `<state-root>/blobs/<aa>/<bb>/<full-hash>` where `aa` and `bb` are the first two hex bytes of the SHA-256. The two-level fan-out keeps any single directory under a few thousand entries even at large scale. RFC-007 will refine this with ref-counts and a GC sweep; Phase 1 stubs both (a blob is never deleted in Phase 1).

6.2 Daemon wiring

The daemon at boot:

1. Opens the state store at `$EASYNET_HOME/state.sled`.
2. Opens the blob store at `$EASYNET_HOME/blobs/`.
3. Calls `pages_ability::register` for every agent in the registry; the registration captures (`state_store`, `blob_store`, `agent_name`) into the handler closure.
4. Boots the receipt index (`state_key` prefix index built on top of the same sled tree).
5. Resumes any in-flight transitions (none in pages: every transition is single-shot, no `Building`-style intermediate state).

6.3 Hub wiring

A separate binary, `easynet-hub`, embeds an axum router:

```
GET /<realm>/<owner>/<slug>          -> human_view
GET /<realm>/<owner>/<slug>/agent     -> agent_view (JSON)
GET /<realm>/<owner>/<slug>/receipts -> receipt history (JSON)
GET /healthz                          -> 200 "ok"
```

The Hub is a daemon client: it has no direct access to the state store. Every view fetch is a `pages.get@v1` invocation against the daemon, with the envelope filled per main §5.4 (caller

URI `principal/human-anon/...`, `caller_context` filled per main §3.6).

Trust boundary. The Hub is allowed to mint `principal/human-anon` callers but not arbitrary `agent/` callers. The daemon validates the caller URI on admission and rejects `agent/` URIs that do not match the Hub’s authorisation token (an out-of-band shared secret in Phase 1; RFC-008 will replace with proper signing).

6.4 Storage layout

```
$EASYNET_HOME/
state.sled/          sled DB
  trees:
    "page::canonical"  key: jcs((realm,owner,slug))
                       value: bincode(PageState)
    "page::receipts"   key: (jcs((realm,owner,slug)),
                       version_be_u64,
                       receipt_id)
                       value: bincode(CanonicalReceipt)
    "page::idx_owner"  key: (owner, slug)
                       value: state_key (back-pointer)
    "page::idx_receipt_id" key: receipt_id
                       value: state_key
  blobs/
    ab/
      f0/
        abf0c1...c8f3  raw bytes
    ...
```

The two indexes (`idx_owner`, `idx_receipt_id`) satisfy main §7.1’s dual-axis enumeration requirement: receipt log enumerable both by `receipt_id` (preserve original EasyNet invocation traceability) and by `state_key` (new object-granularity traceability).

7 Implementation plan

Five PR slices. Each slice is independently reviewable and verifiable; merging the first does not block subsequent work in parallel branches.

7.1 P-A1: BlobStore + StateStore skeletons

Files added. `src/runtime/blob_store/` (`mod.rs` + `fs.rs`); `src/runtime/state/` (`mod.rs` + `store.rs` + `hash.rs` + `receipt.rs`).

Public API.

```
pub trait BlobStore: Send + Sync {
    fn put(&self, bytes: &[u8]) -> Result<[u8; 32]>;           // returns sha256
    fn get(&self, hash: &[u8; 32]) -> Result<Vec<u8>>;
    fn exists(&self, hash: &[u8; 32]) -> Result<bool>;
}

pub trait StateStore<S: StateObject>: Send + Sync {
    fn read(&self, key: &S::Key) -> Result<Option<S>>;
    fn apply(
        &self,
        receipt: &CanonicalReceipt,
        post_state: &S,
    )
```

```

    ) -> Result<()>; // atomic: state mutation + receipt append
    fn list_by_owner(...) -> Result<Vec<S::Key>>;
    fn enumerate_receipts(&self, key: &S::Key)
        -> Result<Vec<CanonicalReceipt>>;
}

pub fn project(state: &dyn StateObject) -> BTreeMap<String, Value>;
pub fn canonicalize_jcs(map: &BTreeMap<String, Value>) -> Vec<u8>;
pub fn canonical_state_hash(state: &dyn StateObject) -> [u8; 32];

```

Tests. JCS conformance (a curated set of RFC 8785 test vectors); SHA-256 round-trip; `StateStore::apply` atomicity under concurrent attempts (RC-INV-2 invariants pinned).

Acceptance. `cargo test -p easynet --lib state` green; benchmarks $\geq 50k$ canonical hashes/sec on commodity hardware (sanity bound, not a strict requirement).

7.2 P-A2: pages.* ability registration + receipts

Files added. `src/runtime/agents/pages_ability.rs`. Touches `src/runtime/agents/mod.rs` to register the namespace.

Behaviour. Every transition listed in §4 is implemented:

1. Validate args against the ability schema.
2. Read current state via `StateStore`.
3. Validate `pre_state_hash` (RC-INV-2).
4. Compute `post_state` and `post_state_hash`.
5. Build `CanonicalReceipt`; sign with owner key (Phase 1: stub Ed25519 signature with a hard-coded keypair per agent; key management deferred to RFC-008).
6. `StateStore::apply` (atomic).
7. Return result.

Tests.

- Round-trip every transition.
- Reject `publish` from `Archived` (must `unpublish` first).
- Reject `create` with malformed slug.
- Reject any transition on `Deleted` (terminal).
- Optimistic concurrency: two `update` attempts racing on the same page slug; one wins, one rejected with `ConcurrencyConflict`.

Acceptance. The ability is callable via `easynet ability invoke`; receipts visible via the existing receipt log surface.

7.3 P-A3: view projection + EAL hints

Files added. `src/runtime/views/` + `src/runtime/eal_hint.rs`.

Behaviour. Pure functions: a `PageState` maps to a `(human_view_response, agent_view_json)` pair plus an EAL hint string. No network, no state mutation.

Tests. Determinism (1000 iterations of identical input yield identical output bytes); `ability_surface` matches §3.3 for every state; CSP header constant.

7.4 P-A4: Hub binary

Files added. `src/bin/easynet-hub.rs` + `src/runtime/hub/`.

Behaviour. axum router, three GET routes, each:

1. Parse path $\rightarrow$ (realm, owner, slug) $\rightarrow$ state_key.
2. Build envelope: caller URI is `principal/human-anon/<ip-hash>/<session-id>` (§7.6); `caller_context` populated from HTTP headers.
3. Invoke `pages.get@v1` on the daemon.
4. Apply view projection (P-A3).
5. Return HTTP response.

Tests. Integration: launch a daemon + Hub in a temp dir, `create+publish` a page, `curl` through the Hub, assert HTML response + correct ETag + correct CSP. `curl` the `/.agent` suffix, assert JSON shape matches §5.6.

7.5 P-A5: agent_view enrichment + receipt history endpoint

Behaviour. Implements the `/.receipts` route: returns the receipt history for a page (canonical receipts only, in version order) as a JSON array. Adds an `eal_hint` field to `agent_view` (already specified in P-A3 but only stubbed there).

Tests. Replay a receipt history client-side using `canonical_state_hash` reconstruction; assert the final hash equals the page's current hash.

7.6 Hub envelope minting

```
fn build_envelope(req: &HttpRequest) -> Envelope {
    let ip    = client_ip(req);
    let salt  = HUB_LOCAL_SALT.read();
    let ip_hash = hmac_sha256(&salt, ip.to_string().as_bytes())
        .iter().take(8)
        .flat_map(|b| format!("{:02x}", b).chars()).collect::<Vec<_>>()
        .collect::<String>();
    let sid = req.cookie("easynet_session")
        .or_else(|| req.header("X-EasyNet-Session"))
        .or_else(|| req.query("session"))
        .unwrap_or_else(|| "-".to_string());

    let caller = format!(
        "easynet:///principal/human-anon/{ip_hash}/{sid}"
    );
    let caller_context = json!({
        "origin": {
            "referrer_url":    req.header("Referer"),
            "referrer_realm":  referrer_origin(req.header("Referer")),
            "direct":          is_direct_navigation(req)
        },
        "agent_class": classify_user_agent(req.header("User-Agent")),
        "headers": {
            "accept_language": req.header("Accept-Language"),
```

```

        "sec_fetch_site":      req.header("Sec-Fetch-Site"),
        "sec_fetch_mode":     req.header("Sec-Fetch-Mode"),
        "sec_fetch_dest":     req.header("Sec-Fetch-Dest"),
        "x_forwarded_for_hash": xff_hash(req)
    }
});
Envelope { caller, caller_context, ... }
}

```

8 End-to-end worked example

A complete trace from `pages.create` through Hub-rendered HTML access and agent URA fetch.

8.1 Setup

```

$ easynet agent add alice --type claude-code
$ easynet-daemon &          # listens on $EASYNET_HOME ipc
$ easynet-hub --bind 127.0.0.1:8787 &

```

8.2 Author and publish

```

$ easynet ability invoke alice.pages.create --args '{
  "slug": "shop-001",
  "manifest": {
    "title": "Mug, ceramic, 12oz",
    "content_type": "text/html",
    "visibility": "public"
  },
  "initial_body": "<h1>Mug</h1><p>$19.99 — ships Tuesday.</p>"
}'

response:
  receipt_id:      "r_a1b2..."
  state_value:     "Draft"
  canonical_state_hash: "0xa3b91f7e..."
  version:         1

$ easynet ability invoke alice.pages.publish --args '{ "slug": "shop-001" }'

response:
  receipt_id:      "r_c3d4..."
  state_value:     "Published"
  canonical_state_hash: "0x4d28ee17..."
  version:         2
  etag:            "TSjuFw0pAvjpAQ=="

```

8.3 Browser fetch via Hub

```

$ curl -i http://127.0.0.1:8787/default/alice/shop-001

HTTP/1.1 200 OK
ETag: "TSjuFw0pAvjpAQ=="
Cache-Control: public, max-age=60, must-revalidate

```

```
Content-Security-Policy: default-src 'self'; script-src 'none'; ...
Content-Type: text/html
Content-Length: 49

<h1>Mug</h1><p>$19.99 — ships Tuesday.</p>
```

The Hub emits an `OperationalReceipt` for the read:

```
{
  "receipt_class": "operational",
  "state_key": ["default", "alice", "shop-001"],
  "transition_id": "pages.get@v1",
  "attempt_id": "att_e5f6...",
  "envelope": {
    "caller": "easynet:///principal/human-anon/9c4e.../sess-abc",
    "caller_context": {
      "origin": { "referer_url": null,
                  "referer_realm": null,
                  "direct": true },
      "agent_class": { "kind": "browser",
                       "user_agent_hash": "a1b2..." },
      "headers": { "accept_language": "en-US",
                   "sec_fetch_site": "none",
                   "sec_fetch_mode": "navigate",
                   "sec_fetch_dest": "document" }
    }
  },
  "observed_at": "2026-04-28T10:14:32Z"
}
```

8.4 Peer agent fetches via URA

```
$ curl http://127.0.0.1:8787/default/alice/shop-001/.agent | jq

{
  "state_type": "easynet.page",
  "state_key": { "realm": "default",
                 "owner": "alice",
                 "slug": "shop-001" },
  "ura": "easynet:///r/default/agent/alice/page/shop-001",
  "canonical": {
    "state": "Published",
    "manifest": { "title": "Mug, ceramic, 12oz",
                  "content_type": "text/html",
                  "visibility": "public" },
    "content_hash": "0x4d28ee17...",
    "visibility": "public",
    "version": 2,
    "etag": "TSjuFw0pAvjpaQ=="
  },
  "ability_surface": ["pages.update@v1", "pages.archive@v1",
                     "pages.delete@v1"],
  "history": {
    "versions": 2,
    "latest_canonical_receipt_id": "r_c3d4...",
    "receipt_log_ura": "easynet:///r/default/agent/alice/page/shop-001/receipts"
  }
}
```

```

},
"actions": null    // simple HTML page; no actions.* namespace.
                   // For an interactive page (form / subscribe / ...) the
                   // owner registers actions.list_for_page etc. and this
                   // becomes a pointer object — see §5.5 actions.
}

```

8.5 Audit replay

A peer fetches the receipt history and replays it locally:

```

$ curl http://127.0.0.1:8787/default/alice/shop-001/.receipts | jq -r '.[].
  post_state_hash'
0xa3b91f7e...    # version 1, after create
0x4d28ee17...    # version 2, after publish

```

The peer:

1. Reconstructs canonical state from each receipt in order.
2. Computes `canonical_state_hash` for each reconstruction.
3. Asserts each computed hash equals the receipt's `post_state_hash`.
4. Asserts the final hash equals the `agent_view`'s `canonical.content_hash` (well, not literally — the comparison is on `canonical_state_hash`, but `content_hash` is one of its inputs).
5. Verifies each receipt's `owner_signature` against `alice`'s published key.

Successful audit means: the peer has cryptographic proof that this page, at version 2, presented exactly this content, signed by Alice, on or before the timestamp in the final receipt. This is the audit story the traditional web cannot tell.

9 Open questions deferred to RFC-006-A v2

1. **HTML sanitisation.** Phase 1 trusts owner-authored HTML entirely (CSP `script-src 'none'` mitigates the worst case but still lets `onerror=`-style attribute injection through). v2 should specify a safe-subset sanitiser run at `publish` time and pin its output hash.
2. **Multi-realm Hubs.** A Hub serving more than one realm needs a routing table; the path scheme `/<realm>/...` already accommodates it but Phase 1 binds a single realm at startup.
3. **Page templates / slot-filling.** Common pages (product catalogue, blog post) repeat structure. A template layer could let owners author manifests + slot data and have the canonical `content_hash` cover the rendered output.
4. **Authenticated humans (`principal/human-auth`).** For private/realm pages that need real user identity, the `principal/human-auth/<provider>/<user-id>` sub-namespace must be specified along with its signature scheme.
5. **Federated reads.** A page owned by node A read through a Hub on node B. The Hub becomes a federation client, not just a daemon client. RFC-006-A v2.

9.1 Coverage check

Main rule	Discharged by	Where
SO-INV-1 (key immutability)	state_key parser refuses key updates	§3
CSH-INV-1 (hash over projection)	project_pages + JCS only	§5
CSH-INV-2 (content as blob)	FsBlobStore; canonical state holds only the hash	§6.4
RC-INV-1 (receipt binding)	every receipt carries (state_key, transition_id, attempt_id)	§4
RC-INV-2 (optimistic concurrency)	ETag check at receipt-validation time	§5.5
TA-INV-1 (transition_id required)	every pages.<verb>@v1 string is validated	§4
VP-INV-1 (ability_surface from state)	derive_ability_surface match block	§5.6
TR-INV-7 (multi-view first-class)	human_view + agent_view both implemented	§5.6
TR-INV-11 (URA-receipt traceability)	/.receipts endpoint + idx_owner index	§6.4, §8
TR-INV-12 (principal/human-anon)	build_envelope mints the URI	§7.6
TR-INV-13 (caller_context)	populated by build_envelope; entered into operational receipts	§7.6, §8

Closing. Pages is the simplest reference machine that exercises the full normative core of stateful EasyNet. Implementing the five PRs above closes Phase 1 of stateful EasyNet and unlocks both the pressure-test machine (**webapp**, Appendix B) and the agent-native web layer that follows.

This document is a Phase-1 implementation specification. Discrepancies with AXON-RFC-006 main are bugs in this document; discrepancies with Appendix B's pressure tests are bugs in either this document or the main body, to be reconciled in alignment.